Understanding Algorithm Efficiency and Scalability

Furqaanahmed Khan

University of the Cumberlands

MSCS 532: Algorithms and Data Structures

Professor Michael Solomon

November 10, 2026

Understanding Algorithm Efficiency and Scalability

Algorithm performance depends on both the input and the choices made during execution. This report compares randomized Quicksort with a first-pivot version and analyzes a hash table that resolves collisions with chaining. The measurements show that a random pivot can protect Quicksort from particularly unfavorable input order, while three-way partitioning reduces work when many keys are equal.

Randomized Quicksort Analysis

Both implementations partition the current subarray into values less than, equal to, and greater than the pivot. The randomized algorithm chooses an index uniformly from that subarray; the deterministic algorithm chooses its first element. An explicit stack replaces recursion, so unbalanced partitions do not exhaust Python's recursion limit. Both modify the supplied list. The equal region is excluded from further sorting, which makes an all-equal input linear rather than quadratic.

For n distinct keys, order them by rank and let Xij indicate whether ranks i and j are compared. They are compared exactly when the first pivot selected from the ranks i through j is one of the two endpoints. Each rank is equally likely to be first, so $E[X_{ij}] = 2/(j - i + 1)$. By linearity of expectation, the expected number of comparisons is the sum of these probabilities over $i < j$. Grouping pairs by distance gives a sum bounded by 2n times the nth harmonic number, which is O(n log n). Linear partition work at each level yields the same asymptotic expected running time (Cormen et al., 2022).

This argument assumes distinct keys and independent uniform pivot selections. With duplicate keys, three-way partitioning eliminates all occurrences of a selected value at once; the expected O(n log n) bound still holds, and all-equal data requires O(n) work. An unlucky

sequence of pivots can still give $O(n^2)$ time. The explicit stack can use $O(n)$ space in the worst case.

Empirical Comparison

The benchmark used Python 3.12.12 on macOS arm64. For each input size and distribution it timed five runs on the same input, reporting the median in milliseconds. Input copying and correctness checks were outside the timed interval. Random input was a seeded permutation; repeated input used ten possible values. Pivot randomness was seeded separately for reproducibility. Timings depend on the interpreter and machine and are not universal constants.

Input	Size	Randomized ms	First pivot ms
random	500	0.371	0.284
random	1000	0.848	0.641
random	2000	1.879	1.599
random	4000	4.103	3.262
sorted	500	0.376	0.624
sorted	1000	0.881	1.774
sorted	2000	1.787	4.791
sorted	4000	4.189	13.333
reverse	500	0.362	5.152
reverse	1000	0.830	22.102
reverse	2000	1.789	93.899
reverse	4000	3.909	387.105
repeated	500	0.212	0.215
repeated	1000	0.325	0.162
repeated	2000	0.348	0.325
repeated	4000	0.706	0.791

At 4,000 elements, reverse-sorted input took 3.909 ms with random pivots and 387.105 ms with first pivots. The latter repeatedly encounters an extreme pivot in this partition scheme, producing quadratic work. On random input, the first-pivot version was faster (3.262 ms versus 4.103 ms), partly because it avoids random-number generation. Sorted input was slower for the first-pivot version, though its partition swaps rearrange later subarrays, so it does not behave exactly like a textbook partition that preserves their order. With only ten distinct values, both versions benefited from grouping equal keys; small timing differences reflect pivot choices and

measurement noise. The experiment uses one generated input per case and five timings, so it does not measure variation across many independent datasets.

Hashing With Chaining

The hash table stores key-value pairs in bucket lists. It accepts integer keys from 0 to $p - 1$, where $p = 2^{61} - 1$, and chooses random a in 1 through $p - 1$ and b in 0 through $p - 1$. The bucket index is $((a \times key + b) \bmod p) \bmod m$. The chosen family limits collision probability for distinct fixed keys; randomizing the function is the central idea of universal hashing (Carter & Wegman, 1979). The table replaces an existing value on insert and raises KeyError for a missing search or deletion.

Let $\alpha = n/m$ be the load factor. Under simple uniform hashing, an unsuccessful search examines an expected α entries, and a successful search examines $O(1 + \alpha)$ entries on average. Deletion first searches the chain and therefore also takes expected $O(1 + \alpha)$. This implementation checks for an existing key before insertion, so insertion likewise takes expected $O(1 + \alpha)$, rather than $O(1)$ for an unconditional prepend. Worst-case operations take $O(n)$ if all keys collide. Simple uniform hashing is an analysis assumption, not a guarantee that every real input is uniformly distributed.

The table doubles its bucket count once the load factor exceeds one and rehashes the entries. An individual resize costs $O(n)$, while geometric growth gives amortized constant rehashing work per insertion. Keeping α bounded supports expected constant-time operations under the hashing assumption, at the cost of more bucket memory. The table does not shrink on deletion; memory can therefore remain allocated after many deletions. Resizing alone cannot prevent collisions from poorly distributed or adversarial keys.

References

Carter, J. L., & Wegman, M. N. (1979). Universal classes of hash functions. Journal of Computer and System Sciences, 18(2), 143–154. https://doi.org/10.1016/0022-0000(79)90044-8

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/